

A hibás folyamat egyszerűen megáll (crash failure) – nem küld hamis üzeneteket, csak leáll. Az üzenetrendszer garantálja, hogy minden elküldött üzenet végül megérkezik, de tetszőlegesen késleltetheti és átrendezheti az üzeneteket. A fő tétel (Theorem 1) kimondja: egyetlen ilyen konszenzus-protokoll sem teljesen helyes – vagyis minden protokollnak létezik legalább egy megengedett végrehajtása, amelyben soha nem születik döntés. A bizonyítás kulsgondolata a bivalens konfiguráció fogalma: olyan állapot, amelyből még mindkét döntési érték (0 és 1) elérhető. A szerzők megmutatják, hogy (1) létezik bivalens kezdeti konfiguráció, és (2) bármely bivalens konfigurációból konstruálható olyan végrehajtás, amely soha nem lép ki a bivalens állapotokból – tehát sosem dönt.

Ez az eredmény nem azt jelenti, hogy a konszenzus a gyakorlatban megoldhatatlan, hanem azt, hogy a megoldásoknak valamilyen extra feltételezéssel kell élniük. Ebből következik, hogy minden működő konszenzus-protokollnak feltételeznie kell valamiféle szinkronitást (vagy véletlenszerűséget). A Bitcoin például felteszi, hogy a csomópontok végül szinkronba tudnak jutni (az üzenetek véges időn belül megérkeznek), és a 10 perces blokk-idő kellően nagy ahhoz, hogy a blokkok elterjedjenek a hálózatban. A Tendermint részlegesen szinkron modellben működik: feltételez egy (ismeretlen, de véges) időkorlátot, amely után az üzenetek biztosan megérkeznek.

A Tendermint $1/3$ arányú csalót képes tolerálni, és két alapvető garanciát nyújt:

- Safety: a lojális validátorok mindig konzisztens blokkokat követnek – soha nem véglegesítenek két ellentmondó blokkot ugyanarra a magasságra.
- Liveness: a rendszer sosem áll le véglegesen, mindig képes új blokkot termelni, feltéve hogy a hálózat végül szinkronba kerül.

A részvételért jutalmat ad a rendszer (tranzakciós díjak és/vagy infláció formájában), amit a validátorok kamatként foghatnak fel a befagyasztott tőkéjük után. Fontos kérdés, hogy mi történik, ha valaki nem szavaz: hogyan lehet megkülönböztetni a szándékos támadót az egyszerűen offline validátortól? A legtöbb rendszer erre a megoldása az, hogy a hosszú ideig inaktív validátorokat fokozatosan büntetik (csökkentik a stake-jüket), de enyhébben, mint az aktívan rosszindulatú viselkedést.

15.5 Proof-of-spacetime

A proof-of-spacetime (PoSt) és az ehhez szorosan kapcsolódó proof-of-space (PoSpace) olyan konszenzus-mechanismusok, amelyek a számítási munka (PoW) helyett a tárterületet használják szűk erőforrásként. Az alapötlet: ha egy résztvevő bizonyítani tudja, hogy egy adott mennyiségű tárhelyet lefoglalt és fenntartja, akkor ezzel „szavazati jogot” kap a konszenzusban. Ez a megközelítés energiatátrétekezőbb a PoW-nál, mivel a merevlemez folyamatos üzemeltetése nagyságrendekkel kevesebb áramot fogyaszt, mint a tömeges hash-számítás.

15.5.1 A proof-of-space alapötlete

A legegyszerűbb konstrukció a következő: megkérünk egy résztvevőt, hogy számoljon ki és tároljon le egy nagy táblázatot (például egy hash függvény inverziós tábláját). Bizonyos időközönként rákérdezzük bizonyos értékekre – ha a résztvevő gyorsan válaszol, azzal bizonyítja, hogy a táblázatot valóban letárolta. Ha rendezve tárolta el az értékeket, gyorsan tudja invertálni a hash függvényt, és ezzel igazolja, hogy elegendő tárhelyet allokalta.

Ez a naiv megközelítés azonban hibás a Hellman-féle time-memory trade-off [86] miatt. Hellman 1980-ban megmutatta, hogy ha N méretű a problémátér (pl. egy N elemű táblázat teljes letárolása), akkor nem szükséges az egész táblázatot eltárolni: $T \cdot S = N$ összefüggés áll fenn, ahol T a számítási idő és S a tárolt adatmennyiség. A két szélsőség: teljes táblázat letárolása ($S = N, T = 1$) vagy teljes újraszámolás ($S = 1, T = N$). A kettő között Hellman megmutatta, hogy $N^{2/3}$ nagyságrendű tárhellyel és $N^{2/3}$ nagyságrendű számításal is megoldható a probléma. Ez azt jelenti, hogy egy naiv proof-of-space rendszerben a résztvevő sokkal kevesebb tárhelyet használhatna a szükségesnél, és a hiányzó adatokat menet közben számolná ki – tehát a bizonyíték nem lenne megbízható.

15.5.2 Hellman trade-off és a DES példa

Hellman az eredeti cikkében a DES titkosítóval illusztrálta a módszert. A DES egy szimmetrikus blokktitkosító, amely a kulcs és a nyílt szöveg párjához egy titkosított szöveget rendel:

$$\text{DES} : \{0, 1\}^{56} \times \{0, 1\}^{64} \rightarrow \{0, 1\}^{64}, \quad c = \text{DES}(k, p)$$

ahol $k \in \{0, 1\}^{56}$ az 56 bites kulcs, $p \in \{0, 1\}^{64}$ a 64 bites nyílt szöveg, $c \in \{0, 1\}^{64}$ pedig a 64 bites titkosított szöveg. A támadó egy rögzített, ismert p nyílt szöveghez tartozó (p, c) párt ismer (választott-nyíltszöveges feltevés), és a célja a k kulcs megtalálása. A kulcster mérete $N = 2^{56}$.

Rögzítsük tehát a p nyílt szöveget. Ekkor a titkosítás egy

$$\text{DES}(\cdot, p) : \{0, 1\}^{56} \rightarrow \{0, 1\}^{64}$$

leképezéssé egyszerűsödik, amely az 56 bites kulcsból 64 bites titkosított szöveget állít elő. Mivel a bemenet és a kimenet dimenziója eltér ($56 \neq 64$), ez a leképezés önmagában nem iterálható. Ezért bevezetünk egy redukciós operátort

$$R : \{0, 1\}^{64} \rightarrow \{0, 1\}^{56},$$

amely a 64 bites titkosított szöveget egy 56 bites kulccsá alakítja (például elhagyja az első 8 bitet). Hangsúlyozzuk, hogy R *nem* fejti vissza a titkosítást: csupán egy rögzített, nyilvános „méretre vágó” szabály. A két leképezés kompozíciójával kapjuk az

$$f : \{0, 1\}^{56} \rightarrow \{0, 1\}^{56}, \quad f(k) = R(\text{DES}(k, p))$$

függvényt. A dimenziók most már illeszkednek: $\text{DES}(\cdot, p)$ a $\{0, 1\}^{56}$ -ból $\{0, 1\}^{64}$ -be, R pedig $\{0, 1\}^{64}$ -ből vissza $\{0, 1\}^{56}$ -ba képez, így f a kulcsteret önmagába viszi. Éppen ez teszi lehetővé, hogy f -et ismételten alkalmazzuk, és így láncokat építsünk.

Előszámítási fázis. Jelölje m a láncok számát és t a láncok hosszát (az egy láncon belül elvégzett f -alkalmazások számát).

1. Generálunk m darab véletlenszerű kiindulási kulcsot: $k_1^{(0)}, k_2^{(0)}, \dots, k_m^{(0)} \in \{0, 1\}^{56}$. Ezek a láncok kezdőpontjai.
2. Minden $i \in \{1, \dots, m\}$ kezdőpontra t -szer egymás után alkalmazzuk az f függvényt, azaz $k_i^{(j)} = f(k_i^{(j-1)})$ minden $j = 1, \dots, t$ indexre. Így m darab, egyenként t hosszú láncot kapunk:

$$k_i^{(0)} \xrightarrow{f} k_i^{(1)} \xrightarrow{f} k_i^{(2)} \xrightarrow{f} \dots \xrightarrow{f} k_i^{(t)}$$

3. A közbülső $k_i^{(1)}, \dots, k_i^{(t-1)}$ elemeket eldobjuk, és csak a láncok elejét és végét tároljuk el: a $(k_i^{(0)}, k_i^{(t)})$ párokat, a végpont $k_i^{(t)}$ szerint rendezve (hogy a keresési fázisban gyorsan, bináris kereséssel vissza tudjuk keresni).

A tárigény tehát $S = 2m$ érték (kezdő- és végpont párok), ami $N = 2^{56}$ -nál lényegesen kisebb, feltéve hogy $m \ll N$.

Keresési fázis. Adott az ismeretlen $k^* \in \{0, 1\}^{56}$ kulcshoz tartozó titkosított szöveg $c^* = \text{DES}(k^*, p) \in \{0, 1\}^{64}$ (ugyanazzal a rögzített p -vel). A keresés:

1. A titkosított szövegből előállítjuk a láncbeli reprezentációt: $y_0 = R(c^*) \in \{0, 1\}^{56}$, majd iterálunk: $y_1 = f(y_0)$, $y_2 = f(y_1)$, $\dots$, $y_j = f(y_{j-1})$. Legfeljebb t darab értéket számolunk ki, $y_0, y_1, \dots, y_{t-1}$ -et (azaz $j = 0, 1, \dots, t-1$), hiszen a tárolt láncok hossza t : ha k^* egyáltalán szerepel egy láncban, akkor legkésőbb $t-1$ lépés után ütközünk a lánc végpontjával.
2. Minden kiszámolt y_j értékre ellenőrizzük, hogy egyezik-e valamelyik tárolt végponttal, $k_i^{(t)}$ -vel (a rendezettség miatt ez bináris kereséssel gyors).
3. Ha $y_j = k_i^{(t)}$ valamely i indexre, akkor k^* feltehetően ebben a láncban van. A gondolatmenet: ha y_j az i -edik lánc végpontjával egyezik, és y_j -t úgy kaptuk, hogy $R(c^*)$ -ra j -szer alkalmaztuk f -et, akkor $R(c^*)$ a láncban j lépéssel a végpont előtt áll, azaz $R(c^*) = k_i^{(t-j)}$. Mivel $R(c^*) = R(\text{DES}(k^*, p)) = f(k^*)$, a k^* kulcs eggyel korábban, a $(t-j-1)$ -edik pozícióban helyezkedik el. Ezért a lánc $k_i^{(0)}$ kezdőpontjából újraszámolva a keresett kulcs

$$k^* = f^{(t-j-1)}(k_i^{(0)}) = k_i^{(t-j-1)}.$$

Ezt a $\text{DES}(k^*, p) = c^*$ egyenlőséggel ellenőrizzük. Ha teljesül, megtaláltuk a kulcsot. Ha nem (téves találat: két különböző lánc összeütközött, mert f nem injektív, így azonos végpontra vezethet különböző kulcsokról), elvetjük és folytatjuk a keresést a következő y_{j+1} értékkel.

4. Ha a teljes $y_0, y_1, \dots, y_{t-1}$ sorozatban egyetlen tárolt végponttal sincs (valódi) egyezés, akkor k^* *nincs benne* egyik tárolt láncban sem, és ez a tábla nem tudja visszafejteni. Ennek oka, hogy m darab t hosszú lánc az ütközések miatt a kulcstérnek jellemzően csak egy részét fedi le, nem mind a 2^{56} kulcsot. A megoldás több, egymástól független tábla használata (különböző R redukciós operátorokkal), amelyek együtt a kulcstér nagy részét lefedik.

A lépésszám szemléletesen: ha k^* a lánc utolsó előtti eleme, akkor $y_0 = R(c^*)$ rögtön egyezik egy végponttal (1 lépés); ha k^* kettővel a vége előtt van, akkor $y_1 = f(y_0)$ -nál találjuk meg (2 lépés); és így tovább, egészen addig, hogy ha k^* a lánc legelső eleme, akkor y_{t-1} -nél lesz ütközés (maximum t lépés). Az összefüggés: a láncok $m \cdot t$ különböző kulcsot fednek le (ideális esetben), tehát ha $m \cdot t = N$, a teljes kulcstér le van fedve. Az idő-memória kompromisszum:

$$T \cdot S = N, \quad \text{ahol } T = t \text{ (keresési idő), } S = m \text{ (tárolt láncok száma)}$$

Az optimális egyensúly $m = t = N^{1/2}$ esetén $T = S = N^{1/2}$, de a láncok közötti ütközések (collisions) miatt a gyakorlatban $m = t = N^{2/3}$ választás adja a legjobb kompromisszumot, $T \cdot S^2 = N^2$ korlát mellett.

A módszer lényege nem a DES-re korlátozódik: valójában egy tetszőleges, nehezen invertálható (egyirányú) g függvény előképének megtalálásáról van szó. A DES példában $g(k) = \text{DES}(k, p)$ volt, de ugyanez az eljárás alkalmazható $g = \text{SHA-256}$ esetén is, azaz amikor egy adott y hash-értékhez keresünk olyan x bemenetet, amelyre $\text{SHA-256}(x) = y$. Ekkor a redukciós operátor a hash kimenetét képezi vissza a bemeneti tartományba, az $f = R \circ \text{SHA-256}$ függvénnyel ugyanúgy láncokat építünk, és a tábla ugyanúgy lehetővé teszi a hash részleges invertálását $N^{2/3}$ nagyságrendű tárhellyel. Pontosan ez a megfigyelés teszi hibássá a naiv proof-of-space konstrukciót: a résztvevőnek nem kell a teljes inverziós táblát letárolnia ahhoz, hogy gyorsan válaszoljon a hash-invertálási lekérdezésekre, így a letárolt tárhely nem bizonyítja megbízhatóan a lefoglalt kapacitást.

15.5.3 Beyond Hellman: modern proof-of-space konstrukciók

A modern proof-of-space protokollok [87] kifejezetten úgy vannak tervezve, hogy ellenálljanak a Hellman-féle trade-off-nak. A cikk fő eredménye a következő: ha két véletlen függvényt kompozícióban alkalmazunk, az invertálás bizonyíthatóan nehezebb lesz, mint egyetlen függvény esetén.

A konstrukció két véletlen függvényt használ: $f : [N] \rightarrow [N]$ és $g : [N] \times [N] \rightarrow [N]$. A kompozíciót a következőképpen definiáljuk:

$$g_f(x) = g(x, x') \quad \text{ahol } f(x) = f(x') \text{ és } x \neq x'$$

Vagyis g_f kiértékeléséhez meg kell találni egy $x' \neq x$ „ütközőpárt”, amelyre $f(x) = f(x')$ – ez az f invertálását igényli. A becslétes bizonyító letárolja f és g_f teljes függvénytábláját (összesen $\approx 2N \log N$ bit), és így bármely y értékre gyorsan megadja az (x, x') párt, amelyre $g_f(x) = y$.

A cikk fő tétele a következő alsó korlátokat bizonyítja az invertáláshoz szükséges S tárhelyre és T lekérdezésre:

Hellman eredeti korlátai:

- Permutációra: $S \cdot T \in \tilde{O}(N)$, azaz $S = T \approx N^{1/2}$ elegendő (egyenlet (1)).
- Véletlen függvényre (Fiat–Naor): $S^2 \cdot T \in \tilde{O}(N^2)$, azaz $S = T \approx N^{2/3}$ elegendő (egyenlet (2)).
- Tetszőleges függvényre: $S^3 \cdot T \in \tilde{O}(N^3)$, azaz $S = T \approx N^{3/4}$ elegendő (egyenlet (3)).

A Beyond Hellman tétel alsó korlátja a g_f kompozícióra:

$$S^2 \cdot T \in \Omega(N^2) \quad (\text{alsó korlát, nem támadás!})$$

Ez azt jelenti, hogy bármely támadónak, aki S bit segédinformációval és legfeljebb T lekérdezéssel akarja invertálni g_f -et, teljesítenie kell $S^2 \cdot T \geq \Omega(N^2)$ -t. Például ha $S = T$, akkor $S = T = \Omega(N^{2/3})$ szükséges – ez megegyezik a véletlen függvények elleni legjobb ismert támadással, tehát a kompozíció nem gyengíti a biztonságot.

A konstrukció továbbá beágyazással (nesting) erősíthető: $k-1$ szintű beágyazás esetén az alsó korlát $S^k \cdot T \in \Omega(N^k)$ -ra javul, ami $S = T = \Omega(N^{k/(k+1)})$ -et ad. Ahogy k nő, ez közelít az $S = \Omega(N)$ vagy $T = \Omega(N)$ ideális korláthoz (vagyis vagy mindent letárolsz, vagy mindent újraszámolsz).

A gyakorlatban ez egy nagyjából 100 GB-os fájl („plot”) generálását jelenti, ami körülbelül 5 perc, ha 400 GB RAM áll rendelkezésre. A kulcskérdés az, hogy mennyi memória és CPU kellhet ahhoz, hogy a letárolás nélkül tudjuk invertálni a táblát. A fenti alsó korlátok garantálják, hogy a menet közbeni kiszámítás lényegesen drágább, mint a tárolás – tehát a proof-of-space biztonságos.

A proof-of-spacetime a proof-of-space időbeli kiterjesztése: nemcsak azt kell bizonyítani, hogy egy adott pillanatban le van foglalva a tárhely, hanem azt is, hogy folyamatosan fenn van tartva egy adott időintervallum alatt. Ez megakadályozza, hogy valaki gyorsan generálja, majd törölje az adatokat, és csak a kérdéses pillanatában tartsa azokat.

A folyamatos tárolás bizonyításának kulcseleme a Verifiable Delay Function (VDF) [88]. A VDF egy olyan függvény, amelynek kiszámítása garantáltan legalább T szekvenciális lépést igényel (nem párhuzamosítható), de az eredmény ellenőrzése gyors. Formálisan: adott egy x bemenet, a VDF kimenetét $y = f^{(T)}(x)$ jelöli, ahol f egy ismételt alkalmazott egyirányú függvény (tipikusan moduláris négyzetemelés: $f(x) = x^2 \bmod N$). A T iteráció szekvenciális jellege miatt a kimenet nem gyorsítható fel párhuzamos hardverrel – pontosan ennyi idő kell a kiszámításához, függetlenül a rendelkezésre álló processzorok számától.

A proof-of-spacetime [89] protokollban a VDF-et arra használják, hogy időbélyegeket fűzzenek a tárolt adatokhoz: a résztvevőnek rendszeres időközönként le kell futtatnia egy VDF-et a tárolt adatokon, és az eredményt be kell nyújtania bizonyítékként. Mivel a VDF kiszámítása a megadott T idő alatt nem rövidíthető le, a résztvevő nem tudja az adatokat törölni és az ellenőrzés pillanatában újragenerálni – a VDF kimenete bizonyítja, hogy az adatok az egész időintervallum alatt rendelkezésre álltak.

15.6 Chia Network

A Chia Network [90] a proof-of-space és a proof-of-time (VDF) kombinációját alkalmazza konszenzus-mechanizmusként. Az alapötlet az, hogy a felhasználók nagy mennyiségű fel nem használt lemezterülettel rendelkeznek, amelyet a blokklánc biztosítására lehetne használni anélkül, hogy jelentős energiát pazarolnának – szemben a PoW-val, ahol speciális hardver és hatalmas villamosenergia-fogyasztás szükséges.

15.6.1 A proof-of-space protokoll

A Chia proof-of-space protokollja két fázisból áll. Az első fázisban (inicializálás, „plotting”) a bizonyító P létrehoz egy F fájlt („plot”), amelynek mérete N . Ez az inicializálási folyamat kvázilineáris N -ben, tehát a bizonyítónak jelentős számítási munkát kell végeznie egyszer, a fájl létrehozásakor. A hitelesítő V ezzel szemben csak egy kis állapotot tárol.

Egy naiv konstrukció az lenne, hogy V megad egy véletlen függvényt $f : [N] \rightarrow [N]$, és P -nek ki kell számítani az f összes értékét és rendezve letárolnia. Ekkor egy kérdésre (y értékre) P gyorsan megtalálná az $f^{-1}(y)$ értéket a rendezett táblából. Ez a megközelítés azonban nem ad megfelelő biztonsági garanciát a Hellman trade-off miatt.

A Chia által használt konstrukció [87] ezt úgy oldja meg, hogy két egymásba ágyazott függvényt (f és g) alkalmaz. A bizonyító invertálja f -et és letárolja, majd kiszámítja g -t, amelynek argumentumába f inverzét is bele kell építeni. Az ellenőrzés során V elküld egy y értéket, és P -nek vissza kell adnia az x és x' értékeket, amelyek igazolják a tárolást. A g függvényt előre irányban sem könnyű kiszámítani adott x -re, mert ehhez invertálni kell f -et – tehát a Hellman-féle rövidítés nem alkalmazható hatékonyan.

15.6.2 A két fázis biztonsági garanciái

A második fázisban (bizonyítási fázis) V ismételten kihívást intéz P -hez, hogy igazolja: még mindig tárolja az F fájlt. A biztonsági követelmény kimondja, hogy egy csaló bizonyító P' , aki csak egy F' méretű fájlt tárol (ahol $|F'| \ll N$), vagy elbukik a bizonyítás során, vagy jelentős számítási kapacitást kell befektetnie – ideális esetben legalább annyit, amennyibe az F fájl teljes tárolása kerülne. Ha megpróbálja rekonstruálni F -et a korábbi kommunikáció alapján, az ugyanolyan költséges, mint ha egyszerűen tárolta volna.

15.6.3 Short Range Replotting Attack

A Chia rendszerben ismert támadási vektor a short range replotting attack: a támadó elkezd a plotting folyamatot egy signage point kiadása után, és befejezi a plot létrehozását az infusion point előtt, majd törli a plotot. Mivel a signage point és a plot ID együtt határozza meg a plot filtert, a támadó mindig tud olyan plot ID-t választani, amely átmegy a 512-es filteren – ezzel effektíve 512 plot-nyi (≈ 51 TiB) tárhelyet szimulál egyetlen géppel. A Chia eredeti konszenzusa (egy signage point per 10 perces sub-slot) sebezhetővé tette a rendszert erre a támadásra: elég volt 10 percnél gyorsabban plotolni. Az új konszenzus 64 signage pontra növelte a sűrűséget (9,375 másodpercenként egy), és az infusion point-ot 3–4 signage pontra helyezte a signage point után. Ez azt jelenti, hogy a támadó számára a minimális konzisztens időablak 28,125 másodperc ($3 \times 9,375$ s), a hálózati késleltetést is figyelembe véve a gyakorlatban ≈ 25 másodperc alatt kellene plotot generálni. Fontos megjegyezni, hogy ez nem klasszikus támadás (a „támadó” nem tud csalni a hálózattal), hanem a rendszert proof-of-work jellegűvé alakítaná – a Chia tervezésének célja pont ennek megakadályozása volt.

15.7 Proof-of-coverage

A proof-of-coverage (PoC) a Helium hálózat [91] egyedi konszenzus-mechanizmusa, amely a fizikai világ egy mérhető tulajdonságát – a rádiófrekvenciás (RF) lefedettséget – használja a konszenzus alapjául. Míg a PoW számítási munkát, a PoS tőkét, a PoSpace tárhelyet igényel, addig a PoC azt bizonyítja, hogy egy eszköz (Hotspot) valódi vezeték nélküli lefedettséget biztosít egy adott földrajzi területen.

Az algoritmus az RF jelek fizikai terjedési tulajdonságaira épül:

- Az RF jelnek korlátozott fizikai terjedési tartománya van.
- A jelerősség a távolság négyzetével fordítottan arányos (inverse square law): $P_r \propto 1/d^2$.
- Az RF jel a fénysebességgel terjed, tehát a terjedési késleltetés elhanyagolható a hálózati késleltetéshez képest.

A PoC protokoll a következőképpen működik: a hálózat véletlenszerűen kiválaszt egy Hotspot-ot „challenger” (kihívó) szerepre. A kiválasztás az előző blokk hash-éből származtatott pszeudovéletlen értéken alapul. Ez nem teljesen manipulálhatatlan – elméletileg a blokkgyártó különböző tranzakció-összetételeket próbálgatva („grinding”) befolyásolhatná a hash-t –, de a manipuláció költsége (blokkjutalom elvesztése, késleltetés) nagyságrendekkel meghaladja az egyetlen PoC challenge megnyeréséből származó hasznot, így gazdaságilag nem racionális. A challenger egy titkosított kihívást (challenge) küld egy másik Hotspot-nak („challengee”, a bizonyító). A bizonyítónak RF jellel kell válaszolnia, amelyet

a környező Hotspot-ok („witnesses”, tanúk) fognak és jelentenek a hálózatnak. A tanúk által jelentett jelerősség (RSSI) és a válaszidő alapján a hálózat eldönti, hogy a bizonyító valóban a megadott helyen üzemel-e, és valódi lefedettséget biztosít-e.

A PoC biztonsága azon alapul, hogy az RF jelterjedés fizikai törvényeit nem lehet szoftveresen hamisítani: ha két Hotspot túl messze van egymástól, a jel nem éri el a másikat; ha túl közel vannak (pl. ugyanabban a szobában összezárva), a jelerősség-minta nem felel meg a valós városi/vidéki környezetnek. A hálózat statisztikai módszerekkel és gépi tanulással detektálja a csalási kísérleteket (pl. több Hotspot azonos helyen, hamis GPS-koordináták, RF jel továbbítása interneten keresztül).

A Helium hálózat a PoC-ot jutalmak kiosztására használja: a Hotspot-ok HNT (Helium Network Token) kriptovalutát kapnak a lefedettség biztosításáért és az adatforgalom továbbításáért. Ez ösztönzi a résztvevőket, hogy valódi, hasznos vezeték nélküli infrastruktúrát építsenek – szemben a legtöbb más blokklánccal, ahol a konszenzus-mechanizmus önmagában nem termel a fizikai világban hasznos mellékterméket.

16 eCash

1982-ben David Chaum publikálta a *Blind Signatures for Untraceable Payments* [92] című tanulmányát, amely az első konkrét javaslat volt kriptográfiai eszközökkel megvalósított digitális pénzre. Chaum már ekkor felismerte, hogy a digitális fizetési rendszerek elterjedése elkerülhetetlen, és hogy ezeknek a rendszereknek óriási hatásuk lesz mind a legális, mind az illegális pénzmozgásokra.

A motiváció egyértelmű volt: ha valakinek nyomon tudjuk követni a pénzmozgásait, rendkívül sok információ derül ki az életéről. A közlekedési szokásai, a szállodák ahol megszáll, az éttermek és szórakozóhelyek amelyeket látogat, az élelmiszerek amelyeket vásárol, a gyógyszerek amelyeket szed – mindez egy teljes profilt rajzol ki az egyénről. A hagyományos banki rendszerekben ez az információ elvileg elérhető, de a készpénz használata eddig biztosított bizonyos fokú anonimitást.

Chaum felismerte, hogy a digitális pénz megjelenésével ez az anonimitás elvész, ha csak nem tervezzük be tudatosan kriptográfiai védelmet. Ugyanakkor a teljes anonimitás is problémás: hogyan bizonyítjuk a pénzmossást, az adóelkerülést, hogyan lépünk fel a feketepiacok ellen? Chaum válasza egy új kriptográfiai primitív volt, amely lehetővé teszi, hogy a normál tranzakciók anonim maradjanak, de szükség esetén (bírói végzéssel) a személyazonosság felfedhető legyen.

16.1 RSA Blind Signatures

Chaum rendszerének három alapvető követelménye van: (1) egy harmadik személy ne tudja lekövetni, hogy kinek történt átutalás, mikor és mennyi; (2) mégis lehessen bizonyítani a fizetés tényét, ha vita merül fel; (3) ha egy digitális bankjegy „elveszik” (lemásolják), azzal ne lehessen kétszer fizetni. Chaum megoldása a vak aláírásra (blind signature) épül, amelyet az RSA kriptográfiai rendszer segítségével valósít meg.

Az RSA vak aláírás protokollja három szereplőt tartalmaz: a bankot (aláíró), a fizetőt (aki a bankjegyet kapja) és a kedvezményezettet (akinek fizetnek). A bank RSA kulcspárja (e, d, n) , ahol e a publikus, d a titkos kitevő, n a modulus. A protokoll a következő lépésekből áll:

1. A fizető generál egy véletlen üzenetet m (ez lesz a bankjegy sorszáma), és választ egy véletlen r blinding faktort, amelyre $\gcd(r, n) = 1$.
2. A fizető kiszámítja a vakított üzenetet:

$$m' = m \cdot r^e \bmod n$$

Ezt küldi el a banknak aláírásra. A bank nem tudja kiolvasni m -et, mert az r^e faktor elfedi.

3. A bank aláírja a vakított üzenetet a titkos kulcsával:

$$s' = (m')^d \bmod n = m^d \cdot r^{ed} \bmod n = m^d \cdot r \bmod n$$

és visszaküldi s' -t a fizetőnek. A bank ezzel 1 dollárt von le a fizető számlájáról.

4. A fizető eltávolítja a blinding faktort:

$$s = s' \cdot r^{-1} \bmod n = m^d \bmod n$$

Az s érték egy érvényes RSA aláírás m -re, amelyet a bank készített – de a bank nem tudja, hogy melyik m -hez tartozik (mert csak m' -t látta).

5. Fizetéskor a fizető átadja az (m, s) párt a kedvezményezettnek, aki a bank publikus kulcsával ellenőrzi: $s^e \equiv m \pmod{n}$.
6. A kedvezményezett beváltja a bankjegyet a banknál. A bank ellenőrzi az aláírást és egy adatbázisban rögzíti m -et, hogy ne lehessen kétszer felhasználni (dupla-költés elleni védelem).

A protokoll biztonsága három tulajdonságon alapul. Az anonimitás abból fakad, hogy a bank a vakított m' értéket látta, nem m -et – tehát amikor m -mel találkozik a beváltáskor, nem tudja összekapcsolni a korábbi kibocsátással. A hamisíthatatlanság az RSA probléma nehézségéből következik: m^d -t csak a bank tudja kiszámítani. A dupla-költés elleni védelem pedig az m értékek nyilvántartásán alapul.

A rendszer lehetővé teszi az adóztatást és a pénzmosás elleni fellépést is: a bank dönthet úgy, hogy gyanús tranzakciókat nem vált be, és a számla-bankjegy összekapcsolás bírói végzéssel feloldható. A forgalomban lévő pénz mennyisége pontosan nyomon követhető a kibocsátott és beváltott bankjegyek alapján.

17 Proof-of-work

A proof-of-work fogalmát Cynthia Dwork és Moni Naor vezette be 1993-ban a *Pricing via Processing or Combatting Junk Mail* [93] című tanulmányukban, jóval a Bitcoin megjelenése előtt. Az eredeti motiváció nem a kriptovaluta, hanem az e-mail spam elleni védekezés volt.

17.1 A probléma és a megoldási megközelítések

A kérdés egyszerű: hogyan lehet megakadályozni, hogy valaki tömeges kéretlen e-maileket küldjön? Dwork és Naor három lehetséges megközelítést azonosított: (1) fizetőssé tenni az e-mail küldést (mikrofizetés); (2) törvénybe iktatni a használati szabályokat; (3) számítási munkát követelni minden elküldött e-mailhez. A harmadik megoldás a legelegánsabb, mert nem igényel sem központi hatóságot, sem jogi keretrendszert – tisztán technikai.

Az ötlet: minden e-mail elküldéséhez a küldőnek meg kell oldania egy közepes nehézségű számítási feladatot, amelyet a fogadó gyorsan ellenőrizni tud. Egy normál felhasználó, aki naponta néhány e-mailt küld, nem veszi észre a számítási költséget; de egy spammer számára, aki milliókat küldene, a kumulált számítási költség gazdaságilag értelmetlenné teszi a spammelést.

17.2 Követelmények a számítási feladattal szemben

Dwork és Naor a következő követelményeket fogalmazta meg a felhasználandó függvénnyel szemben:

- A nehézség legyen állítható egy paraméterrel (hogy a hardver fejlődéséhez lehessen igazítani).
- A kiszámítás legyen közepes nehézségű (pl. ≈ 10 másodperc), de az ellenőrzés legyen triviálisan gyors (pl. $\approx 0,01$ másodperc).
- Ne lehessen a megoldást előre kiszámolni és eltárolni (time-memory trade-off ellenállóság) – ezért az üzenet tartalmától kell függnie a feladatnak.
- A rendszergazda (vagy egy megbízható fél) számára legyen könnyítés lehetséges („trapdoor”), hogy szükség esetén feloldást adhasson.

A feladat az üzenettől, az időbélyegtől és a címzettől függ. Formálisan: adott egy h hash függvény, és a feladat az, hogy a küldő találjon egy x értéket, amelyre

$$y = h(m || t || d || x)$$

teljesít egy adott feltételt, ahol m az üzenet (vagy annak hash-e), t az időbélyeg, d a címzett azonosítója. Az ellenőrzés triviális: a fogadó kiszámítja h -t és ellenőrzi a feltételt, valamint hogy az időbélyeg nem túl régi.

17.3 Javasolt számítási feladatok

Dwork és Naor három konkrét számítási feladatot javasolt:

1. Négyzetgyök számolása modulo egy nagy prím p : adott y , keressük azt az x -et amelyre $x^2 \equiv y \pmod{p}$. A nehézség a prím méretével állítható. Ha ismerjük p faktORIZÁCIÓJÁT (trapdoor), a gyök gyorsan kiszámolható.
2. Feltört aláírás-sémák felhasználása: olyan digitális aláírás algoritmusok, amelyekről kiderült, hogy nem biztonságosak (pl. az Ong–Schnorr–Shamir séma [94], amelyet Pollard tört fel [95]), de az aláírás-hamisítás éppen a kívánt nehézségű feladatot adja.

3. Általánosabban: bármely probléma, amelyre létezik hatékony ellenőrzés (NP-beli), de a megoldás keresése kontrollálhatóan nehéz.

Ez a tanulmány fektette le az elméleti alapját annak, amit ma proof-of-work-nek hívunk. Adam Back 1997-ben javasolt Hashcash rendszere (formálisan 2002-ben publikálva) és végül Satoshi Nakamoto Bitcoinja mind erre az alapgondolatra épül: a számítási munka költségesé teszi a visszaélést, miközben az ellenőrzés olcsó marad.

18 Hashcash

A Hashcash-t Adam Back javasolta 1997-ben (formálisan 2002-ben publikálta [77]) azzal a céllal, hogy védje az internetes erőforrásokat elárasztásos (DoS) támadások és spam ellen. A Dwork–Naor féle elméleti keretet Back egy konkrét, egyszerű és elegáns konstrukcióvá alakította, amely később a Bitcoin proof-of-work mechanizmusának közvetlen elődjévé vált.

18.1 A Hashcash cost-function

A Hashcash a következő tulajdonságokkal rendelkező cost-function-t definiál:

- Nehéz kiszámolni (a küldőnek számítási munkát kell végeznie).
- Könnyű leellenőrizni (a fogadó egyetlen hash-számítással verifikálja).
- Bármely harmadik fél le tudja ellenőrizni, nem csak a címzett.
- Nincs trapdoor (rövidítés): senki sem tudja megkerülni a számítást, még a rendszer tervezője sem. Ez fontos különbség a Dwork–Naor javaslathoz képest, ahol trapdoor létezett.
- Nem interaktív: a küldőnek nem kell előzetesen kommunikálnia a fogadóval (nincs „kérdés” a szervertől).
- Nem újrahasznosítható: minden megoldás egyedi, egy adott üzenethez és időponthoz kötött.

18.2 A Hashcash protokoll

A Hashcash egy részleges hash-ütközést keres. A küldő a következő fejléctet állítja össze:

$$\text{header} = \text{ver} : \text{bits} : \text{date} : \text{resource} : \text{rand} : \text{counter}$$

ahol ver a verziószám, bits a megkövetelt nehézség (hány vezető nulla bit), date az időbélyeg, resource a címzett azonosítója (pl. e-mail cím), rand egy véletlenszerű érték, counter pedig a próbálgatott nonce. A küldő addig növeli a counter értékét, amíg

$$\text{SHA-1}(\text{header}) \text{ első } k \text{ bitje} = 0$$

nem teljesül, ahol k a bits paraméter értéke. Átlagosan 2^k próbálkozás szükséges a megfelelő counter megtalálásához.

Az ellenőrzés triviális: a fogadó kiszámítja egyszer a hash-t és megnézi, hogy az első k bit valóban nulla-e, valamint ellenőrzi, hogy az időbélyeg nem túl régi és a resource a saját címe. Az újrahasznosítás megakadályozása érdekében a fogadó nyilvántart egy adatbázist a már felhasznált Hashcash tokenekről (double-spending adatbázis), és az időbélyeg lejáratát után törli őket.

A Hashcash tehát az első széles körben ismert, nem interaktív, trapdoor-mentes proof-of-work rendszer. Satoshi Nakamoto a Bitcoin whitepaper-ben explicit módon hivatkozott rá, és a Bitcoin bányászat lényegében a Hashcash mechanizmus általánosítása: a blokk-fejléc hash-ének kell egy target érték alá esnie, ami ugyanaz az elv, mint a k vezető nulla bit keresése.

19 Bitcoin

A Bitcoin [45] az előző fejezetekben tárgyalt építőelemek szintézise: Chaum digitális pénz ötlete, Lamport elosztott konszenzusa, a bizánci hibatűrés, és Back Hashcash proof-of-work mechanizmusa. Satoshi Nakamoto 2008-ban egyesítette ezeket egy központi hatóság nélküli elektronikus pénzrendszerrel, amelynek első tranzakciói 2009 januárjában történtek.

19.1 A Bitcoin rendszer összefoglalása

A rendszer működését a korábbi fejezetekben már részletesen tárgyaltuk (digitális aláírásokkal megvalósított tranzakciók a 13. fejezetben, proof-of-work a 14. és 17. fejezetben). Itt a Bitcoin specifikus tervezési döntéseit foglaljuk össze:

- A tulajdonjogot ECDSA digitális aláírások (secp256k1 görbén) igazolják.
- A dupla-költés problémáját elosztott főkönyvvel oldja meg: a leghosszabb lánc (amelyhez a legtöbb proof-of-work tartozik) a kanonikus történet.
- A blokkok SHA-256 hash-sel vannak láncolva, a bányászok Hashcash-szerű részleges hash-ütközést keresnek.
- A nehézséget 2016 blokkonként (≈ 2 hét) állítják, hogy a blokkidő ≈ 10 perc maradjon.
- Minimális strukturális követelmények: bárki szabadon csatlakozhat és kiléphet a hálózathoz, nincs regisztráció, nincs engedélyezés.

19.2 Biztonsági modell

A Bitcoin biztonsági garanciája a következő: ha a becsületes csomópontok rendelkeznek a számítási kapacitás többségével, a támadó esélye, hogy utólag felülírja a tranzakciós történetet, exponenciálisan csökken a megerősítések (confirmations) számával. Nakamoto megmutatta, hogy ha a támadó a hashrate q hányadával rendelkezik (ahol $q < 0.5$), akkor annak a valószínűsége, hogy z blokknyi hátrányt behozzon:

$$P(\text{sikeres támadás}) = \left(\frac{q}{1-q} \right)^z$$

Ez exponenciálisan csökken z -vel, ami indokolja a gyakorlatban használt 6 megerősítéses szabályt.